

- **FLASK INTERFACE**

- 7 actors name/id (up to)
- 3 writers name/id (up to)
- 3 directors name/id (up to)
- Budget (dollars, impute if missing)
- Genre (checkbox menu for 28 different genres, selection of up to 3 max), check 'genre_feature/genre_scores.csv' for all genre names

- **INPUT/DATA PROCESSING**

- **Actor score**

- Data file: 'actors_feature/actor_scores_and_features.csv'
 - Field 'actor' is actor id from the IMDb database, you can match the actor id with 'nconst' in name.basics.tsv.gz dataset on imdb to get actor names, or vice versa
 - 'Actor_score' is the last column in the file, and has the individual actor scores
- Movie 'actor_score' is the average of the 'actor_score' of 7 main actors

- **Director quality**

- Data file: 'directors_quality/director_quality.csv'
 - Use 'director_id' to get director names from the name.basics.tsv.gz dataset on IMDb (same as actors)
 - 'director_quality' is the column with the individual 'director_quality' values
- Movie 'director_quality' is the average of 'director_quality' of the 3 leading directors for a movie.

- **Writer quality**

- Data file: 'writer-quality.csv'
 - Use 'writer_id' to get writer names from the name.basics.tsv.gz dataset on IMDb (same as actors)
 - 'Writer_quality' is used to compute the feature
- Movie 'writer_quality' is the average of 'writer_quality' of the 3 leading writers for a movie.

- **Genre**

- Data file: 'genre_feature/genre_scores.csv'
 - Column 'genre' has the genre name, which should map 1 to 1 with the drop-down menu (or checkboxes) on the Flask interface
 - 'Avg_rating' is the individual genre score
- Feature 'genre_score' is the average of up to three genre 'avg_ratings'

- **Budget**

- Use the input value directly from the FLASK interface if available
- Otherwise, impute based on genre

- Data file 'budget_feature/genre_budgets.csv'
 - Field 'genre' is the genre name
 - 'Avg_budget' is the average budget for the genre
- 'Budget' is imputed as the average of the up to 3 movie genres

- **MODELS**

- The Random Forest model that i trained is saved as
'/random_forest/rf_model.joblib' and the scaler used in training is
'/random_forest/rf_scaler.joblib' which you should use to make the prediction
based on the input
- To use those files you need to use python library 'joblib', I wrote a demo on how
that could work in 'random_forest/rf_demo.py'